UNIVERSIDADE POLITÉCNICA PORTALEGRE | Escola Superior de Tecnologia, Gestão e Design

Trabalho Prático – A Ponte de Sentido Único (Win32, C)

Sistemas Operativos

Projeto de sincronização de threads: exclusão mútua, coordenação e equidade

---

Neste projeto simula-se o tráfego numa ponte onde os carros só podem circular num sentido de cada vez. Cada carro é uma *thread* e a ponte é um recurso partilhado cujas regras têm de ser garantidas por mecanismos de sincronização da API Win32. O desafio não é só impedir acidentes: é também evitar que algum carro fique à espera para sempre.

1. Cenário e regras

Uma ponte estreita liga duas margens, **Norte** e **Sul**. Os carros chegam de ambos os lados em instantes aleatórios, esperam a sua vez, atravessam e saem. O sistema tem de respeitar quatro condições em qualquer execução:

Condições	Regra
C1. Sentido único	Nunca há carros de sentidos opostos na ponte ao mesmo tempo
C2. Capacidade	Há no máximo `MAX_PONTE` carros na ponte (por omissão, 3)
C3. Equidade	Nenhum carro espera indefinidamente, mesmo com tráfego constante do sentido oposto
C4. Progresso	O sistema nunca fica em *deadlock*: se a ponte está livre e há carros à espera, algum entra

Carros do mesmo sentido podem atravessar em simultâneo, até à capacidade máxima.

2. Objetivos de aprendizagem

- Traduzir as regras de um problema de concorrência em condições e verificá-los automaticamente durante a execução.
- Usar `Mutex` e `Semaphore` e perceber o que cada um resolve melhor.
- Aplicar o padrão de coordenação de grupo (o primeiro de um grupo fecha a porta, o último abre-a).
- Identificar e demonstrar *starvation* e corrigi-la.
- Comparar o débito e a equidade de várias soluções para o mesmo problema e discutir o compromisso entre ambos.

3. Estrutura do programa

O programa chama-se `ponte.exe` e recebe os parâmetros da simulação:

```
ponte.exe <n_norte> <n_sul> <chegada_ms> <travessia_ms> [semente]
```

- `<n_norte>`, `<n_sul>`: número de carros que chegam de cada lado.
- `<chegada_ms>`: intervalo máximo, em milissegundos, entre chegadas de carros do mesmo lado (valor aleatório entre 0 e este máximo).
- `<travessia_ms>`: tempo que um carro demora a atravessar.
- `[semente]`: semente do gerador aleatório, para repetir execuções.

Cada carro é descrito pela estrutura seguinte, e a sincronização fica **exclusivamente** dentro das duas funções de acesso à ponte:

```
typedef enum { NORTE = 0, SUL = 1 } Sentido;

typedef struct {
    int     id;
    Sentido sentido;
    DWORD   chegada;      /* instante de chegada, em ms */
    DWORD   entrada;      /* preenchido por ponte_entrar */
    DWORD   saida;        /* preenchido por ponte_sair */
} Carro;

void ponte_entrar(Carro *c);   /* bloqueia até ser seguro entrar */
void ponte_sair(Carro *c);
```

A função da *thread* de cada carro é fixa: espera o tempo de chegada, chama `ponte_entrar`, faz `Sleep(travessia_ms)` e chama `ponte_sair`. O aluno só altera o interior de `ponte_entrar`, `ponte_sair` e o seu estado partilhado.

Verificador de condições

O aluno implementa um verificador independente da solução. Em *ponte_entrar* e *ponte_sair* regista-se, **através de operações atómicas de incremento e decremento**, quantos carros de cada sentido estão na ponte. Se C1 ou C2 forem violados, o verificador incrementa um contador de violações e imprime o estado. No fim, o programa mostra o número de violações, que deve ser **zero** em todas as fases corretas.

4. Fases do projeto

Fase	O que fazer	Cotação
1. Sem controlo	`ponte_entrar` e `ponte_sair` apenas atualizam os contadores. Executar 20 vezes com tráfego intenso e registar as violações de C1 e C2	2
2. Ponte exclusiva	Um único `Mutex` garante que só um carro está na ponte. Verificar que é correta e medir o débito (carros por segundo)	2

Fase	O que fazer	Cotação
3. Sentido e capacidade	Permitir vários carros do mesmo sentido, até `MAX_PONTE`, com um semáforo de capacidade e um contador por sentido protegido por `Mutex` (o primeiro carro de um sentido ocupa a ponte, o último liberta-a)	6
4. Equidade	Demonstrar a *starvation* da fase 3 com tráfego contínuo de um só lado e corrigi-la, por exemplo com um torniquete (um semáforo que todos os carros atravessam à chegada) ou com lotes alternados. Medir o efeito	5
5. Ambulância com prioridade	Acrescentar uma ambulância com prioridade absoluta, que passa à frente de todos os carros à espera sem quebrar C1 nem C2. Medir o impacto na espera dos restantes carros	3
Relatório e qualidade	Organização do código, comentários, relatório	2
Total		**20**

5. Cenários de teste obrigatórios

Cada fase correta (2 a 5) deve passar todos os cenários, com 10 sementes diferentes por cenário.

Cenário	Parâmetros indicativos	O que testa
T1. Tráfego equilibrado	30 Norte, 30 Sul, chegadas até 200 ms, travessia 300 ms	Funcionamento normal
T2. Um único sentido	40 Norte, 0 Sul	Capacidade máxima e ausência de bloqueios sem carros opostos
T3. Rajada assimétrica	60 Norte, 5 Sul, chegadas até 20 ms	*Starvation* do sentido minoritário
T4. Travessia curta	40 + 40, travessia 5 ms	Corridas nas transições de sentido
T5. Carro único	1 Norte, 0 Sul	Casos limite

6. Métricas

O programa escreve, no fim, um resumo por sentido com:

- tempo de espera de cada carro (da chegada à entrada): média, máximo e percentil 95;
- débito total, em carros por segundo;
- número de violações dos invariantes.

Apresente no relatório uma tabela que compare as fases 2, 3, 4 e 5 em T1 e T3. Os valores são a mediana de 5 execuções por cenário. Discuta o compromisso entre débito e equidade: uma ponte mais justa é também mais lenta?

7. Questões de análise

1. Na fase 1, que escalonamento concreto de duas *threads* viola C1? Porque é que a taxa de violações aumenta com `travessia_ms`?
2. Na fase 3, porque é que o último carro de um sentido tem de libertar a ponte, e o que correria mal se fosse o contador a ser lido sem `Mutex`?
3. Porque é que a fase 3 pode causar *starvation*? Explique com um exemplo numérico tirado dos seus dados do cenário T3.
4. Que garantia de ordem dá (ou não dá) um `Semaphore` ao libertar *threads* bloqueadas? Como afeta a equidade da sua solução?
5. Dê um exemplo de *deadlock* que ocorreria se invertesse a ordem de aquisição de dois objetos de sincronização na sua solução.
6. Como garantiu na fase 5 que a ambulância não viola C1 nem C2? Que carros foram prejudicados e em quanto tempo?

8. Lista de verificação

☐ Zero violações de C1 e C2 em todos os cenários, nas fases 2 a 5.

☐ A fase 1 deixa evidências arquivadas de violações reais.

☐ Nenhuma espera ativa: carros à espera não consomem CPU.

☐ Nenhum `Sleep` é usado como mecanismo de sincronização (apenas para simular tempos).

☐ O tempo máximo de espera do sentido minoritário em T3 é limitado e muito inferior ao da fase 3.

☐ Todos os `HANDLE` são fechados e todos os valores de retorno são verificados.

☐ O programa termina sozinho quando todos os carros atravessaram.

9. Entrega e avaliação

- **Código:** `fase1.c` … `fase5.c` e um `LEIAME.txt` com instruções que entender adequadas para a correcta execução do programa.
- **Relatório (PDF, máximo 8 páginas):** desenho da solução e dos objetos de sincronização, evidências da fase 1, tabela de métricas, respostas à secção 7 e dificuldades encontradas.
- **Formato:** um ficheiro `SO_PROJ1_<n_aluno>.zip` entregue no PAE até à data indicada.
- **Grupos:** até 2 alunos. Ambos têm de saber explicar qualquer parte do código numa discussão oral.
- **Integridade:** declare no relatório qualquer uso de ferramentas de IA. Cópias entre grupos implicam a anulação do trabalho dos envolvidos.